

PROFESSIONAL PROJECT REPORT

Kavya HRMS Human Resource Management System

End-to-end project documentation covering architecture, modules, role-based flow, API structure, business rules, deployment readiness, and future enhancement scope.

PREPARED FOR
Kavya HR 360

PREPARED ON
29 July 2026

PROJECT TYPE
React + Spring Boot HRMS

Table Of Contents

1. Executive Summary	10. Backend API Catalogue
2. Project Identification	11. Database Design
3. Business Objective And Scope	12. Business Rules
4. Technology Stack	13. Security And Session Management
5. Repository Structure	14. Configuration And Deployment
6. System Architecture	15. Testing Coverage
7. User Roles And Access Matrix	16. Risks, Recommendations, And Future Scope
8. Application Flow	17. Conclusion
9. Module-Wise Functional Description	

This report was prepared from the local source code under `React Project` . It reflects the observed implementation, folder structure, routing, APIs, models, services, and configuration files available in the project workspace.

1. Executive Summary

Kavya HRMS is a full-stack Human Resource Management System designed to centralize daily HR operations, workforce administration, attendance, leave, payroll, projects, tasks, asset tracking, announcements, support tickets, interviews, and user management. The system provides role-based dashboards for Super Admin, HR Manager, Project Manager, Team Lead, and Employee users.

The frontend is implemented as a React 18 single-page application using Vite, React Router, Bootstrap, and Remix Icon. The backend is implemented with Spring Boot, MongoDB, WebSocket support, mail integration, session handling, and REST APIs. The project is suitable for internal enterprise HR operations and can be positioned as an MNC-style internal HR 360 platform.

FRONTEND React 18 + Vite	BACKEND Spring Boot 4.1	DATABASE MongoDB	PRIMARY DOMAIN HR Operations
------------------------------------	-----------------------------------	----------------------------	--

Key Capabilities

- Role-based dashboards and navigation for all major HR personas.
- Employee master data management with profile photo upload support.
- Attendance recording, team attendance view, and 9 PM Asia/Kolkata auto-checkout process.
- Leave request, approval, recommendation, and leave balance summary workflows.
- Payroll generation from employee, attendance, and leave data with paid payslip controls.
- Project and task assignment workflows with attachment handling.
- Asset inventory, asset assignment, return, and employee asset request workflows.
- Announcements, notifications, support tickets, scheduled interviews, and settings management.

Executive Conclusion

The application has a mature operational scope and a clear full-stack architecture. It already includes core enterprise workflows, backend persistence, seed data, API tests, runtime configuration, and deployment files. Recommended next steps are production hardening, secret management, formal RBAC enforcement at backend level, CI/CD, observability, and replacing any remaining local fallback data with API-only flows for enterprise-grade consistency.

2. Project Identification

Attribute	Details
Project Name	Kavya HRMS / Kavya HR 360
Application Type	Full-stack HRMS web application
Frontend Package	kavya-hrms , version 1.0.0
Backend Artifact	com.kavya:hrms-backend:0.0.1-SNAPSHOT
Backend Description	Kavya HRMS Spring Boot Backend
Default Frontend Port	5173 through Vite development server
Default Backend Port	8080 through SERVER_PORT configuration fallback
Default Database	mongodb://localhost:27017/hrmsdb
Prepared Date	29 July 2026

3. Business Objective And Scope

Objective

The objective of Kavya HRMS is to provide a centralized platform that reduces manual HR work, improves employee service delivery, and gives managers real-time visibility into teams, attendance, leave, payroll, tasks, assets, and communication.

In Scope

- Authentication, session validation, password reset, and forced password change flows.
- Employee lifecycle data, profile records, HR user records, and access roles.
- Attendance self-service and team attendance management.
- Leave request creation, review, approval, and leave summary calculation.
- Payroll generation, mark-paid workflow, payslip generation eligibility, and payroll records.
- Project assignment, project teams, task assignment, task status updates, and attachments.
- Asset inventory, assignment, return, employee asset requests, and request status changes.
- Announcements, notifications, support tickets, interviews, settings, and seeded defaults.

Out Of Scope For Current Implementation

- External identity provider integration such as SSO, SAML, or OAuth.
- Advanced statutory payroll engine with country-wise tax slabs and compliance filings.
- Centralized audit log UI in the current visible routing structure, although an audit model exists.

- Production CI/CD, monitoring dashboards, and infrastructure-as-code are not fully present in the local repository.

4. Technology Stack

Layer	Technology	Usage
Frontend Framework	React 18.3.1	Component-based UI for dashboards, pages, forms, tables, and workflow screens.
Build Tool	Vite 8.0.16	Frontend development server, build, proxy configuration, and production bundle generation.
Routing	React Router DOM 6.28.1	HashRouter-based route handling with protected role routes.
UI Foundation	Bootstrap 5.3.3, Remix Icon 4.6.0	Responsive layout, component styling, and iconography.
Backend Framework	Spring Boot 4.1.0	REST API, service layer, interceptors, scheduled services, and application lifecycle.
Database	MongoDB with Spring Data MongoDB	Document storage for employees, users, sessions, attendance, leaves, payroll, tasks, projects, assets, and settings.
Security Utilities	Spring Security Crypto	Password and security-related backend utility dependency.
Communication	Spring Mail, SendGrid configuration, WebSocket	Email notifications, credential/reset emails, interview notifications, and settings broadcast support.
Backend Tests	Spring Boot Starter Test	Controller and service test coverage.
Deployment	Netlify frontend config, backend Dockerfile	Frontend static deployment and containerized backend build path.

5. Repository Structure

```

React Project/
  Frontend/
    src/
      App.jsx
      main.jsx
      styles.css
      components/
      layouts/
      pages/
      routes/
      utils/
      data/
    package.json
    vite.config.js
  backend/
    src/main/java/com/kavya/hrms/
    config/

```

```
controller/  
dto/  
model/  
repository/  
service/  
websocket/  
HrmsBackendApplication.java  
src/main/resources/application.properties  
src/test/java/com/kavya/hrms/  
pom.xml  
Dockerfile  
PROJECT_FLOW.md  
netlify.toml
```

The structure follows a conventional full-stack separation. The frontend owns routing, layouts, pages, browser-side session state, API utility wrappers, and fallback caches. The backend owns persistent data models, repositories, controllers, services, interceptors, scheduled tasks, and configuration.

6. System Architecture

Kavya HRMS uses a layered architecture. The browser loads the React SPA. Authenticated users move through role-specific protected routes and call backend APIs through a centralized frontend API client. Spring Boot receives requests, validates session tokens using Authorization bearer tokens or cookies, applies forced password change checks, executes controller/service logic, and persists documents in MongoDB.

```

User Browser
|
v
React SPA - HashRouter, ProtectedRoute, DashboardLayout
|
v
Central API Client - headers, timeout, error normalization
|
v
Spring Boot REST Controllers - /api/*
|
v
Interceptors - session validation, forced password change
|
v
Service Layer - payroll, leave, email, notification, attendance jobs
|
v
Spring Data Repositories
|
v
MongoDB Collections + Upload Storage
  
```

Frontend Architecture

- `App.jsx` defines role dashboards, public routes, protected route groups, and session bootstrap.
- `ProtectedRoute.jsx` checks local session state, required password change state, and allowed role mapping.
- `DashboardLayout.jsx` provides the common shell with sidebar, header, content area, login toast, and periodic access refresh.
- `Sidebar.jsx` provides persona-specific menus for admin, HR, team lead, project manager, and employee users.
- `api.js` centralizes API requests, timeout handling, auth headers, role headers, employee headers, and payload normalization.

Backend Architecture

- Controllers expose REST resources for domain modules.
- Repositories provide MongoDB CRUD and query operations.
- Services encapsulate business logic such as payroll generation, payroll validation, leave summary, notifications, email, and attendance auto-checkout.
- Config classes manage CORS, static upload serving, Mongo indexes, session interceptors, mail settings, and seed data.
- WebSocket classes support live settings broadcast capabilities.

7. User Roles And Access Matrix

Access Role	App Role	Default Dashboard	Key Permissions / Modules
Super Admin	admin	/admin/dashboard	Users, employees, attendance, leave, payroll, announcements, settings, projects, tasks, assets, support.
HR Manager	hr	/hr/dashboard	Employees, users, attendance, leave approval, payroll, announcements, assets, projects, tasks, scheduled interviews, support, settings.
Project Manager	projectManager	/project-manager/dashboard	Projects, team view, tasks, team attendance, leave review, self attendance, self payroll, assets, support.
Team Lead	teamLead	/team-lead/dashboard	Team dashboard, team attendance, task assignment, leave review, self attendance, payslip, assets, support.
Employee	employee	/employee/dashboard	Self attendance, leave request, payslip, profile, asset requests, tasks, announcements, support.

Seed Login Accounts

Role	Email	Employee Identity	Default Password
Super Admin	admin@gmail.com	ADMIN-001 , Admin Kavya	admin123
HR Manager	hr@gmail.com	HR-001 , Meera Nair	hr123
Team Lead	teamlead@gmail.com	KV003 , Kabir Khan	teamlead123
Project Manager	manager@gmail.com / projectmanager@gmail.com	KV004 , Isha Patel	manager123
Employee	employee@gmail.com	KV001 , Aarav Sharma	employee123

For production, seeded demo credentials should be disabled or rotated and passwords should be stored only as secure hashes.

8. Application Flow

End-To-End User Flow

1. User opens the Kavya HRMS frontend.
 2. React application initializes through Vite bundle and HashRouter.
 3. Public route shows login, forgot-password, or change-password page.
 4. User submits credentials.
 5. Frontend calls POST /api/auth/login.
 6. Backend validates user and creates a session token.
 7. Token and user profile details are saved in session storage/local state.
 8. User is redirected to the dashboard mapped to the resolved role.
 9. Protected routes validate local session and allowed role.
 10. DashboardLayout loads sidebar/header and refreshes users/employees periodically.
 11. Module pages call REST APIs through apiRequest.
 12. Backend verifies session, runs business logic, persists data, and returns JSON.
 13. UI renders latest rows, summary cards, modals, toasts, validation messages, or error messages.
 14. Logout or session expiry clears token and redirects user to login.

Authentication And Session Flow

Login Page

-> authenticateUser()

-> POST /api/auth/login

-> AuthController

-> AppUserRepository + AuthSessionRepository

-> LoginResponse with token, role, employee identity

-> startSession()

-> Role dashboard redirect

-> ProtectedRoute checks session on every protected route

Role Routing Flow

Role	Route Group	Main Landing Route
Admin	/admin/*	/admin/dashboard
HR	/hr/*	/hr/dashboard
Team Lead	/team-lead/*	/team-lead/dashboard
Project Manager	/project-manager/*	/project-manager/dashboard
Employee	/employee/*	/employee/dashboard

Data Flow

UI Form / Action

-> Component state validation

- > apiRequest(path, options)
- > Headers: Authorization, X-Kavya-Access-Role, X-Kavya-User-Id, X-Kavya-Employee-Id
- > Spring controller
- > Service rules
- > Repository save/query
- > MongoDB document
- > JSON response
- > UI refresh / cache update / toast / error display

9. Module-Wise Functional Description

Module	Primary Users	Functional Summary
Dashboard	All roles	Role-specific summaries for HR, admin, employee, team lead, and project manager personas. Uses dashboard, employee, attendance, leave, announcement, project, task, and settings APIs.
Authentication	All users	Login, session validation, session touch, logout, forgot password, reset password, and forced first-login password change support.
Employees	Admin, HR	Employee listing, profile data entry, edit, delete, duplicate handling, profile photo upload/removal, and sync with user records.
User Management	Admin, HR	User account listing, creation through bulk sync, role assignment, status management, and account deletion.
Attendance	Employees, HR, Team Lead, Project Manager, Admin	Self attendance, team attendance, attendance record creation, employee-specific attendance lookup, attendance status, and backend auto-checkout.
Leave Management	Employees, Team Lead, Project Manager, HR, Admin	Leave request creation, recommendation, approval, rejection, status notes, and current employee leave summary.
Payroll	Admin, HR, self-service users	Payroll record listing, monthly generation, paid marking, payslip generation after payment, employee-specific payroll, and salary calculation based on attendance and leave.
Projects	Admin, HR, Project Manager, Team Lead	Project CRUD, team lead mapping, project members, milestones, progress, and team-lead-specific project lookup.
Tasks	Admin, HR, Project Manager, Team Lead, Employee	Task creation, assignment, status updates, assignee/assigner filters, project mapping, attachment upload, and delete workflow.
My Team	Team Lead, Project Manager	Team member overview, project-team visibility, team tasks, attendance summary, and member-level drill-down context.
Assets	Admin, HR, Project Manager, Team Lead, Employee	Asset inventory, page data, my assets, assignment records, return process, employee asset requests, and request approval/status changes.
Announcements	Admin, HR, all viewers	Announcement create, update, delete, bulk seed, category, priority, status, and role-oriented viewing.
Notifications	All roles	Header notifications with unread state, mark-read, clear, and role/user filtering.
Support	All roles	Support ticket creation, screenshot/upload data, ticket detail view, assignment, status, and resolution workflow.

Scheduled Interviews	HR, Admin	Candidate interview scheduling, resume details, interview round/mode, status, today view, and share-with-admin operation.
Settings	Admin, HR with limited matrix	Company name, timezone, working hours, week off, payroll cutoff, departments, designations, holidays, leave types, role permissions, and payroll settings.
Profile	All roles	User and employee profile view/edit context using employee and user APIs.

10. Backend API Catalogue

Area	Base Endpoint	Supported Operations Observed
Authentication	/api/auth	Login, forgot password, reset password, change password, session validate, session touch, session delete.
Users	/api/users	List users, delete user, bulk save/sync.
Employees	/api/employees	List, create, bulk save, update, delete, upload profile photo, remove profile photo.
Attendance	/api/attendance	List, team payload, employee lookup, create/update record, bulk save.
Leave	/api/leaves	List, create, bulk save, update status/details, current employee leave summary.
Payroll	/api/payroll	List, employee payroll, month/year payroll, generate, create record, mark paid, payslip, bulk save.
Projects	/api/projects	List, team-lead projects, create, bulk save, update, delete.
Team Lead	/api/team-lead	Team lead project lookup.
Tasks	/api/tasks	List, filter assigned-to, assigned-by, owner, assignee-name, create, bulk save, update, status patch, attachment upload, delete.
Assets	/api/assets	List, page data, my assets, create, bulk save, update, delete.
Asset Assignments	/api/asset-assignments	List, create assignment, return asset, delete assignment.
Asset Requests	/api/asset-requests	List, create request, update request status.
Announcements	/api/announcements	List, create, bulk save, update, delete.
Notifications	/api/notifications	List by role/user, mark read, delete one, delete all by user.
Support	/api/support	List, detail, create JSON/multipart ticket, status update.
Interviews	/api/interviews	List, detail, create, update, delete, share with admin, today view.
Dashboard	/api/dashboard	Admin summary, employee summary, monthly attendance, today's interviews.
Settings	/api/settings	Read and update default system settings.
Health / Root	/ , /health	Basic deployment status and health responses.

11. Database Design

The backend uses MongoDB document collections through Spring Data repositories. The model package indicates the following major collections and entities.

Collection / Entity	Purpose	Representative Fields
app_users	Application login users and access identity.	userId, email, password/passwordHash, role, employeeId, status, reset token, mustChangePassword, systemUserIdentityKey.
auth_sessions	Active authentication sessions.	token, userId, email, role, employeeId, lastLogin, createdAt, lastSeenAt.
employees	Employee master data and HR profile.	employeeId, name, contact, joining, managerId, department, jobTitle, accessRole, IDs, address, bank, packageAmount, credential email flags.
attendance_records	Daily attendance records.	employeeId, date, checkIn, checkOut, workedHours, totalHours, status, remarks.
leave_requests	Employee leave applications and approvals.	employeeId, leave type, dates, days, status, recommendation details, final action details.
payroll_records	Monthly payroll results and payslip data.	employeeId, month, year, earnings, deductions, netSalary, payableDays, lopDays, bank, tax identifiers, status, paidDate.
projects	Project and team mapping.	name, manager, teamLeadId, teamLeadName, teamMembers, milestones, dates, progress, status.
tasks	Task assignment and tracking.	title, assignee, assigner, priority, dueDate, status, projectId, attachments.
assets	Asset inventory.	assetCode, assetName, category, brand, serialNo, purchaseDate, assignment dates, status, employee assignment, condition, location.
asset_assignments	Asset dispatch and return history.	assetId, assetCode, employeeId, assignedDate, dueDate, returnDate, condition, status, dispatchedBy.
asset_requests	Employee asset service requests.	employeeId, assetId, requestType, description, screenshot, status, resolution, handledBy.
announcements	Company communication.	title, body, category, priority, postedAt, postedBy, ownerRole, status.
support_tickets	Internal support requests.	ticketId, employee, category, priority, status, screenshot metadata, assignedTo, resolution.
interviews	Interview scheduling and tracking.	candidate details, position, CTC, resume data, schedule, interviewer, status, sharedWithAdmin.
system_settings	Company configuration and policy settings.	companyName, timezone, workingHours, weekOff, payrollCutoff, departments, designations, holidays, leaveTypes, permissionMatrix, payrollSettings.

Indexes

- `app_users.systemUserIdentityKey` is configured as a unique sparse index.
- Asset indexes exist for asset code, assigned employee, and status.
- Asset assignment indexes exist for employee/date, asset/date, and asset code/date lookups.
- Asset request indexes exist for employee/date and asset/date lookups.

12. Business Rules

Attendance

- Open attendance records are finalized automatically at 9:00 PM Asia/Kolkata.
- Records from previous dates without checkout are also finalized.
- Worked minutes decide status: 8 hours or more means Present, 4 to 8 hours means Half Day, below 4 hours means Absent.
- Leave-marked attendance is excluded from auto-checkout.

Leave

- Default leave types are Casual Leave 12, Sick Leave 10, Earned Leave 18, and Work From Home 0.
- Current leave summary resolves employee identity from request headers or user mapping.
- Only approved leave requests count as taken leave.
- Remaining balance is total allotted minus approved taken days, never below zero.

Payroll Generation

- Payroll generation runs for selected month and year for all active employees.
- Annual packages of 300000 or more are converted to monthly gross by dividing by 12.
- Attendance and approved leave are summarized before payroll calculation.
- Absent and half-day deductions are based on per-day salary.
- PF is calculated when PF/UAN exists, using 12 percent of 50 percent of monthly gross.
- Gratuity and professional tax are included in deduction summary.

Payroll Validation

- Already paid records cannot be marked paid again.
- Future payroll periods cannot be marked paid.
- Current month mark-paid is disabled after day 15.
- Payslip generation is allowed only for paid records with positive net salary.

Session

- Session token is accepted from Authorization bearer token or `kavyaAuthToken` cookie.
- Session expires after 1 hour from the latest seen timestamp.
- Expired or missing sessions return 401 and clear the auth cookie.
- Public auth endpoints include login, forgot password, reset password, and session delete.

Settings

- Default company settings use Asia/Kolkata timezone.
- Default working hours are 09:00 AM to 06:00 PM.
- Default week off is Sunday.
- Default payroll cutoff is the 25th of every month.

13. Security And Session Management

Implemented Controls

- Protected frontend routes enforce role-level access before rendering pages.
- Backend interceptors enforce authenticated session validation across `/api/**`.
- Forced password change interceptor is registered for API routes.
- API requests include Authorization bearer token, user ID, employee ID, and access role headers.
- CORS is restricted to configurable origin patterns, defaulting to localhost development patterns.
- Uploaded resources are served through a dedicated `/uploads/**` resource handler.
- API client normalizes errors and clears stale session state on unauthorized backend responses.

Security Recommendations

Area	Recommendation	Priority
Password Storage	Ensure all persisted passwords use strong one-way hashing and remove plaintext fallback/password fields in production.	High
RBAC	Mirror frontend permission checks on backend endpoints so users cannot bypass access through direct API calls.	High
Secrets	Store SMTP, SendGrid, database, and deployment secrets only in environment variables or a secret manager.	High
Audit	Use the existing AuditLog model to capture sensitive changes such as payroll, role, employee, and settings updates.	Medium
Session	Consider refresh-token rotation, device/session management UI, and stricter cookie security in production HTTPS.	Medium

14. Configuration And Deployment

Local Development

Component	Command / Configuration
Backend	<code>cd backend then mvn spring-boot:run</code>
Frontend	<code>cd Frontend then npm run dev</code>
Backend URL Override	<code>VITE_BACKEND_URL=http://localhost:8081</code> when backend runs on a non-default port.
MongoDB	Default local URI: <code>mongodb://localhost:27017/hrmsdb</code>
Uploads	Served from local <code>uploads</code> directory through <code>/uploads/**</code> .

Frontend Build And Hosting

- Vite production build command: `npm run build`.
- Netlify configuration uses `Frontend` as base, `dist` as publish directory, and routes all paths to `/index.html`.
- Vite dev server runs at `127.0.0.1:5173` and proxies `/api` and `/uploads` to the backend.

Backend Container

- Multi-stage Docker build uses Maven with Eclipse Temurin 17 to package the backend JAR.
- Runtime image uses Eclipse Temurin 17 JRE.
- Dockerfile exposes port `10000` and starts `java -jar app.jar`.
- Production should align platform port variables with Spring `SERVER_PORT` configuration.

Mail Configuration

- SMTP is controlled through environment variables such as `SMTP_HOST`, `SMTP_PORT`, `SMTP_USERNAME`, `SMTP_PASSWORD`, and `SMTP_FROM`.
- SendGrid is configurable through `SENDGRID_API_KEY`, sender variables, API URL, and timeout variables.
- Email features include password reset, employee welcome/credential emails, and interview notification emails.

15. Testing Coverage

The repository includes backend tests and frontend rule/credential scripts. The backend tests focus on controllers and business services.

Test Area	Files Observed
Authentication	AuthControllerTest.java
Employees	EmployeeControllerTest.java
Projects	ProjectControllerTest.java
Tasks	TaskControllerTest.java
Support	SupportControllerTest.java
Interviews	InterviewControllerTest.java , InterviewNotificationEmailServiceTest.java
Settings	SystemSettingsControllerTest.java
Payroll	PayrollValidationServiceTest.java , Frontend/test-payroll-rules.js
Credentials	Frontend/test-credentials.js

Recommended QA Expansion

- Add frontend component/integration tests for login, role redirects, employee CRUD, leave approval, payroll generation, and asset request flows.
- Add backend authorization tests per role for each sensitive endpoint.
- Add file upload tests for task attachments, support screenshots, and profile photos.
- Add payroll generation service tests covering absent, half-day, approved leave, no attendance, PF/no PF, and future-month validations.
- Add end-to-end smoke tests for all five roles.

16. Risks, Recommendations, And Future Scope

Category	Observation	Recommendation
Security	Demo credentials and local fallback authentication are convenient for development.	Disable demo fallback in production builds and require secure backend-only authentication.
Authorization	Frontend role routing is strong for UX, but enterprise systems need backend-enforced role permissions.	Add centralized backend RBAC annotations/services across controllers.
Payroll	Payroll rules are operational but simplified.	Add configurable earning/deduction components, statutory tax rules, audit approval, and payroll lock periods.
Deployment	Frontend Netlify and backend Docker are present, but full infrastructure pipeline is not present.	Add CI/CD, environment separation, health checks, and release rollback process.
Observability	Application logs exist through Spring logging, but central monitoring is not visible in the repo.	Add structured logs, metrics, tracing, error tracking, and alerting for production readiness.
Data Governance	HRMS stores sensitive employee, identity, payroll, and document data.	Add data retention policy, audit trail UI, export controls, backup/restore procedure, and access review.

Future Scope

- Single sign-on with Microsoft Entra ID, Google Workspace, or SAML provider.
- Multi-level approval chains for payroll, leaves, assets, and employee changes.
- Advanced analytics for attrition, attendance trends, project staffing, and payroll forecasting.
- Employee document vault with verification workflow.
- Mobile-first employee self-service experience.
- Manager scorecards and performance review cycles.
- Compliance reports, audit exports, and maker-checker approvals for sensitive records.

17. Conclusion

Kavya HRMS is a comprehensive HR operations platform with a meaningful enterprise scope. It includes a modern React frontend, Spring Boot backend, MongoDB persistence, role-based flows, operational modules, payroll and attendance business logic, file upload support, mail integration, seeded defaults, and a deployable structure.

The project is well positioned as an internal MNC-style HR 360 application. With additional production hardening around security, backend authorization, audit logs, CI/CD, monitoring, and statutory payroll compliance, it can evolve into a robust enterprise-grade HRMS platform.

Final Project Flow Snapshot

```
Employee / HR / Manager / Admin
-> Login
-> Session token generated
-> Role dashboard
-> Sidebar module selection
-> Module page loads data through API
-> Spring Boot validates session
-> Service executes business rule
-> MongoDB stores or returns records
-> UI refreshes with latest operational state
```

Prepared as a professional project report for Kavya HRMS, based on source review of frontend, backend, configuration, services, routes, models, and API mappings available on 29 July 2026.